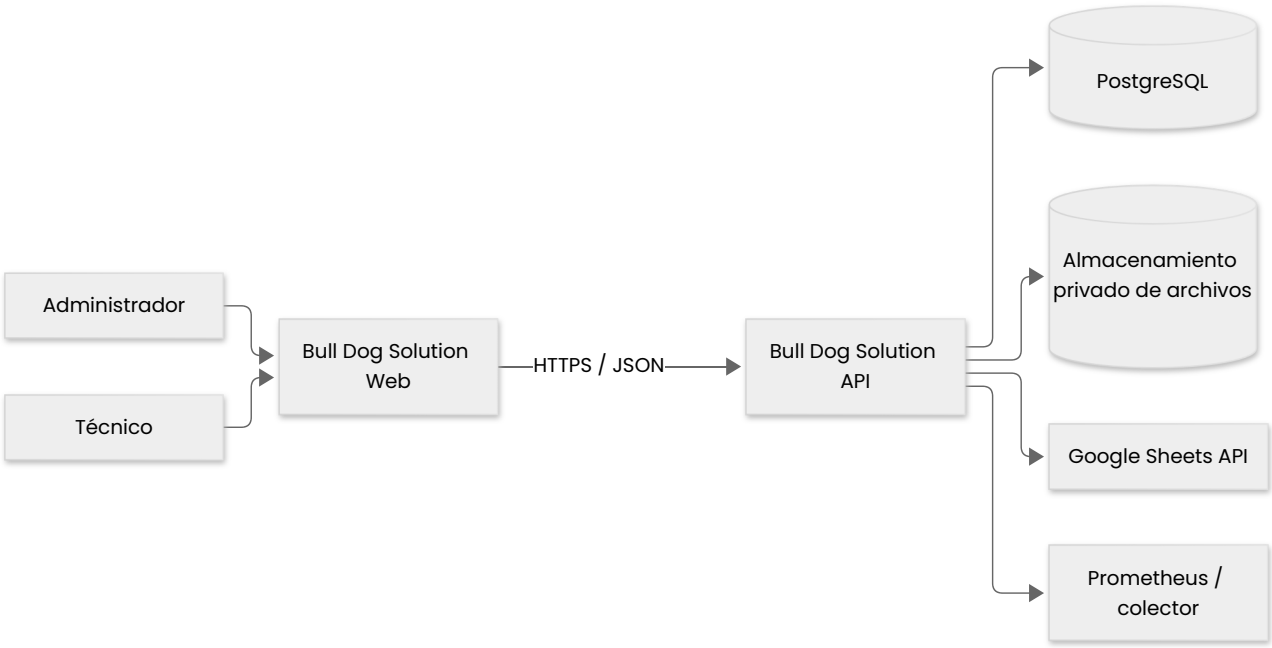
Arquitectura

Resumen

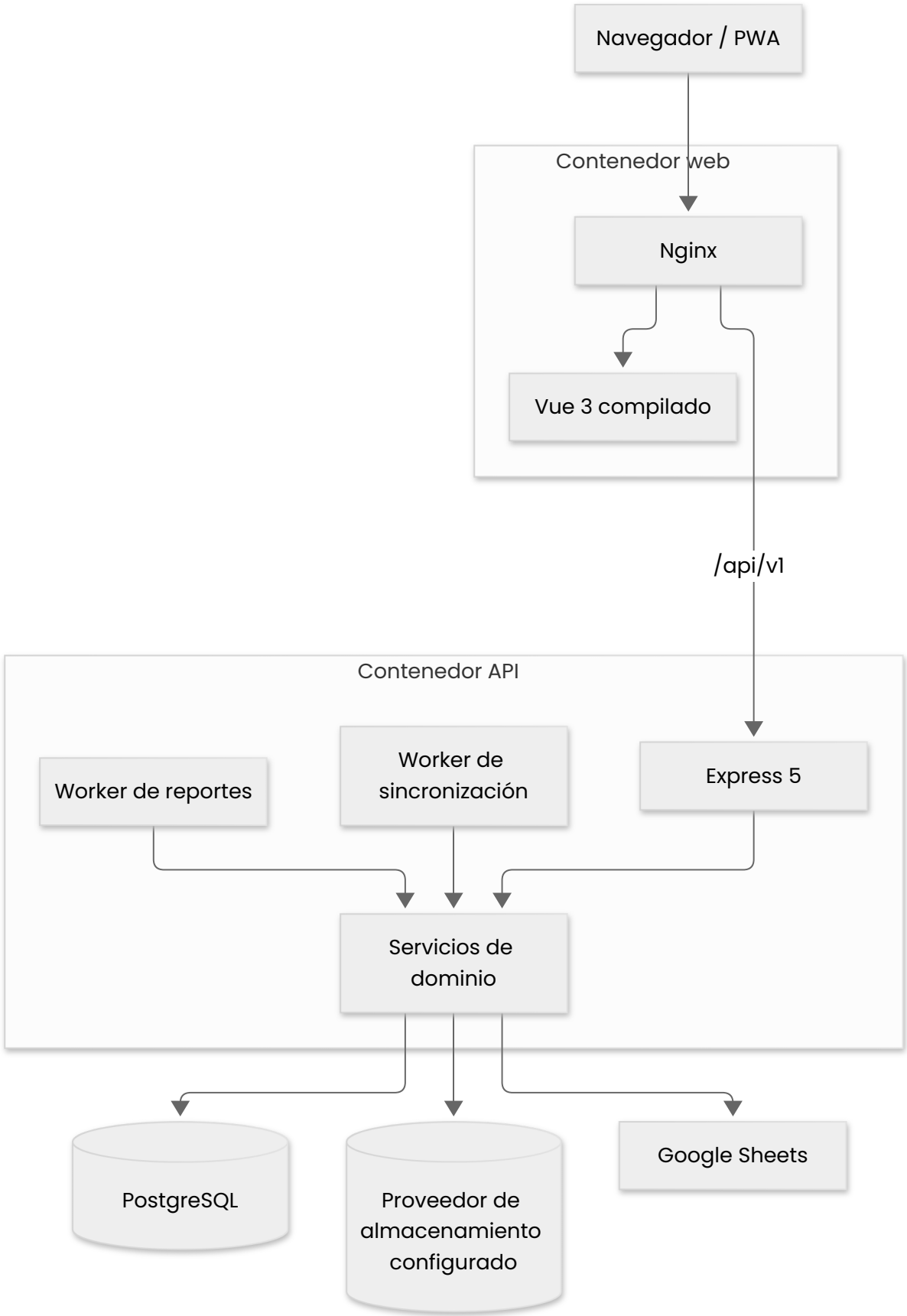
Bull Dog Solution es un monorepo TypeScript con un frontend Vue y una API Express desplegables por separado. La API implementa un monolito modular: autenticación, órdenes, ejecución técnica, catálogos, sincronización, archivos y reportes comparten proceso y base de datos, pero están separados por módulos de dominio.

PostgreSQL conserva el estado transaccional. Google Sheets sigue siendo la fuente externa de clientes y equipos. Google Drive compartido es el proveedor configurado por la IaC de staging para fotografías, firmas y PDFs; el adaptador de almacenamiento de objetos permanece en el código para compatibilidad/otros entornos. PostgreSQL conserva únicamente metadatos y referencias. Staging AWS está eliminado y producción no tiene infraestructura propia.

Diagrama de contexto

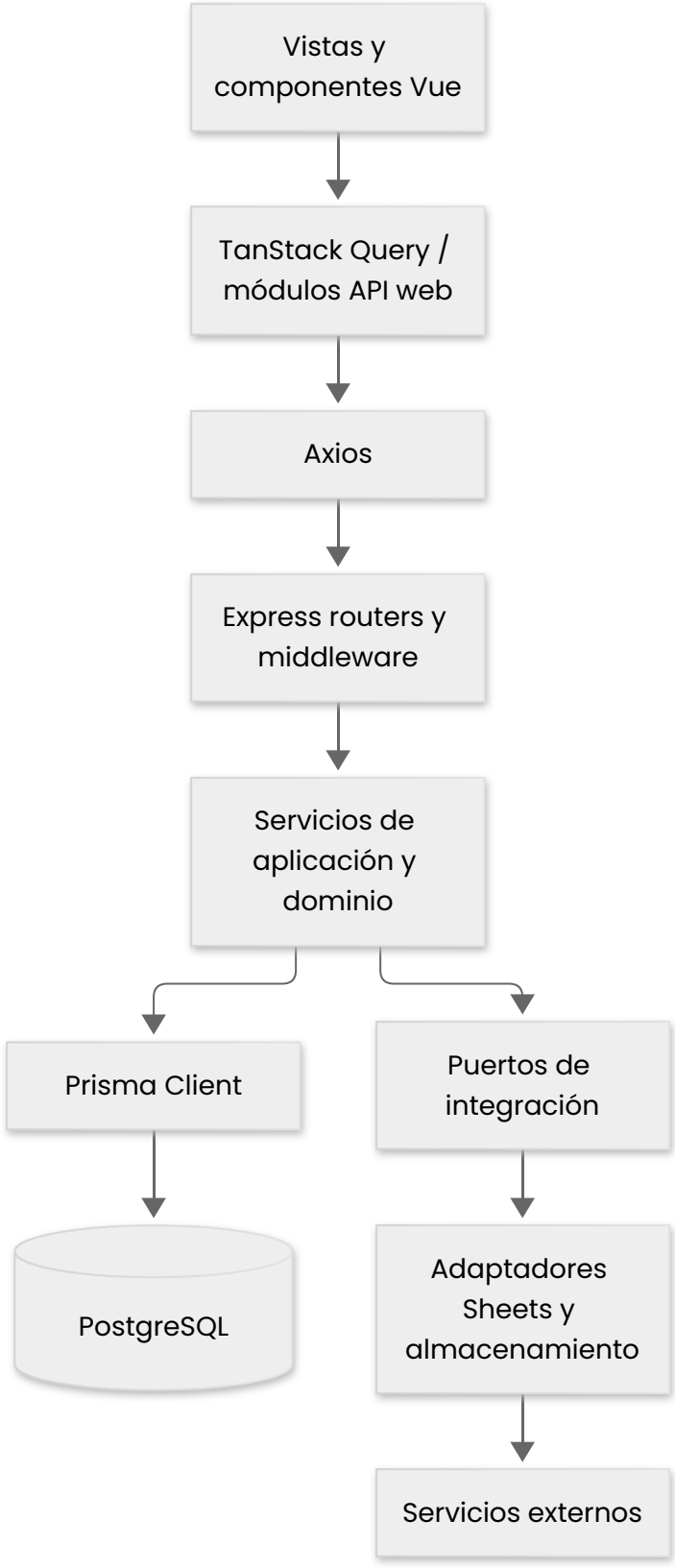


Contenedores desplegables



En AWS, Nginx y Express viven en la misma tarea Fargate y Nginx puede alcanzar la API en `127.0.0.1:3000`. Docker Compose ofrece un proveedor de objetos local mediante MinIO y un volumen persistente; es una configuración de desarrollo, no el modelo arquitectónico del almacenamiento. La topología detallada y la divergencia conocida del proxy local están documentadas en Infraestructura y operación.

Capas lógicas



Responsabilidades

- El frontend coordina navegación, formularios, cámara, firma, estados de carga y consumo HTTP.

- Los routers autentican, autorizan, validan y traducen HTTP; no deben contener reglas de negocio.
- Los servicios validan estados, propiedad, concurrencia y efectos del dominio.
- Prisma implementa persistencia y transacciones.
- Los puertos aíslan SDKs y siguen segregación de interfaces (ISP): `SheetsPort`, `ObjectStoragePort`, `EquipmentCatalogPort`, `ClientCatalogPort`, `CatalogServicePort` y `ReportGeneratorPort`.
- La resiliencia de integración aplica el patrón Decorator con `CircuitBreaker` configurable y telemetría hacia Prometheus.
- `packages/contracts` comparte esquemas Valibot y tipos de entrada entre API y web.

Estructura del monorepo

```
apps/
├── api/              API, workers, integraciones y OpenAPI
└── web/              SPA/PWA Vue
packages/
├── contracts/        DTOs y validaciones compartidas
├── eslint-config/    reglas de lint
└── typescript-config/  configuraciones TS compartidas
prisma/               esquema, migraciones y seed
infra/docker/         imágenes de API/web y Nginx
infra/cdk/            stacks de staging en AWS
infra/scripts/        migración, respaldo, restore, smoke y E2E
.github/workflows/    CI y despliegue continuo de staging
docs/                 requisitos, decisiones y documentación técnica
```

Decisiones arquitectónicas vigentes

- Monolito modular antes que microservicios.
- Catálogos externos con snapshots locales dentro de la orden.
- Escrituras a Sheets mediante outbox persistente y reintentos.
- Almacenamiento privado de evidencias, firmas y reportes detrás de adaptadores de proveedor.
- Reportes PDF generados en segundo plano y versionados mediante abstracción `ReportGeneratorPort`.
- Ciclo de vida y transiciones de orden gobernados por una máquina de estados de dominio explícita (`order-state-machine`).
- Validación de cierre y completitud técnica desacoplada mediante patrón de especificación pura (`order-completeness`).
- Tolerancia a fallos en adaptadores externos (Sheets y almacenamiento) protegida por Circuit Breaker con estados CLOSED, OPEN y HALF-OPEN.
- JWT de acceso en memoria del navegador; refresh token en cookie HTTP-only.

- TanStack Query para estado remoto; Pinia se limita a sesión y estado global de UI.
- Mocks de Sheets, almacenamiento y claves JWT únicamente en desarrollo con valores `development-only`.
- Staging definido como infraestructura inmutable en dos stacks CDK: foundation para estado/red y application para cómputo/entrega.

Estado implementado al 2026-09-30

- El acceso técnico compartido se modela como configuración versionada y auditable; las políticas distinguen ejecución individual de tablero multiorden.
- Cada request protegido resuelve el principal vigente desde PostgreSQL. El JWT identifica la sesión, pero no es la fuente final de actividad ni de rol.
- Las respuestas de detalle de órdenes usan read models explícitos: el detalle administrativo y el detalle operativo tienen selects, mapeadores y contratos separados.
- La web reconcilia la identidad con `/auth/me` al navegar, recuperar foco y actualizar el tablero; las APIs de cuentas técnicas viven fuera del módulo `orders`.
- Las consultas de órdenes, la reprogramación de citas fallidas, la finalización y los medios de ejecución, el almacenamiento de reportes y varios flujos de la web tienen límites dedicados; sus servicios/vistas públicas conservan fachadas y contratos compatibles. El alcance y la deuda residual se detallan en la auditoría de mantenibilidad.
- Evidencias, firmas y reportes usan el proveedor configurado; Google Drive está integrado y el adaptador de objetos se conserva para entornos que lo seleccionen. Cada proveedor mantiene sus protocolos de permisos, reintento y confirmación.
- Las tarifas de órdenes admiten personalización acordada previa y ajuste contable post-cierre mediante `OrderPricingService` y cálculo puro en `pricing.ts`, con precisión arbitraria, auditoría, versiones contables y publicación multilínea en la columna de costo del reporte técnico de Sheets.

La secuencia histórica de implementación y sus commits está en la línea de tiempo del proyecto.